

Task #8 - Wazuh & n8n Integration Automation

SOC Automation Technical Report

Goal

Integrate Wazuh with n8n using webhooks to automate alert handling, risk-based triage, and email escalation.

Prepared by: Suhaib Kashif
Perspective: SOC Analyst with CISA/CISM/CISSP-aligned control thinking
Platform: Wazuh 4.14.6, n8n, Docker, VMware, SMTP Email
Date: August 2026

Confidentiality note: This report documents a controlled laboratory implementation. Test IP addresses from documentation ranges were used for workflow validation. Credentials, SMTP secrets, and passwords are intentionally excluded from the report and from repository-ready configuration artifacts.

Contents

1	Executive Summary	4
2	Task Objectives and Deliverables	4
3	Integration Architecture	4
3.1	Logical Architecture	5
3.2	Network Placement	5
4	Lab Environment and Baseline Validation	5
4.1	Wazuh Manager Status	5
4.2	Wazuh Network Interface	6
4.3	Docker Runtime Validation	6
5	n8n Deployment	7
5.1	Container Deployment	7
5.2	Connectivity Validation from Wazuh to n8n	8
6	Webhook Configuration and Initial Validation	8
6.1	Webhook Design	8
6.2	Manual Test Event	9
6.3	Production Webhook Test from the Wazuh VM	10
7	Wazuh Custom Integration Implementation	11
7.1	Integration Environment	11
7.2	Custom Integration Files	12
7.3	Launcher Script	12
7.4	Python Forwarder Logic	13
7.5	Manual Integration Validation	14
8	Wazuh Integrator Configuration	14
8.1	Change Control and Backup	14
8.2	Troubleshooting the Wazuh Argument Format	15
9	Real Wazuh-to-n8n Alert Delivery	15
10	Unique Automation Use Case	16
10.1	Use Case Definition	16
11	n8n Workflow Design and Logic	16
11.1	Workflow Nodes	17

11.2 Alert Normalization and Classification	17
11.3 False Branch Validation	19
11.4 Final Workflow	19
12 Email Notification Automation	19
12.1 Notification Content	20
13 Multi-Severity Workflow Testing	22
13.1 Testing Method	22
13.2 Test Cases and Expected Results	22
13.3 Execution Evidence	22
14 Results	25
15 SOC, CISA, CISM, and CISSP Considerations	26
15.1 SOC Operations Perspective	26
15.2 CISA-Oriented Control and Audit Perspective	26
15.3 CISM-Oriented Risk and Incident Management Perspective	26
15.4 CISSP-Oriented Security Architecture Perspective	26
16 Findings	27
17 Recommendations	27
18 Conclusion	27
A Evidence Index	28
B Repository-Ready Artifacts	29
C Reference Configuration Snippets	29
C.1 Wazuh Integration Block	29
C.2 Example Controlled Alert Structure	29
C.3 Operational Validation Commands	30

List of Figures

1	Logical architecture of the Wazuh-n8n alert automation workflow.	5
2	Wazuh Manager active and running on the Ubuntu virtual machine, with the host address visible for baseline validation.	6
3	Wazuh VM network configuration showing the active <code>ens33</code> interface and IPv4 address.	6
4	Docker and Docker Compose versions verified on the Windows host.	7
5	n8n web interface successfully running after deployment.	7
6	Docker container listing showing the n8n container and published port 5678.	7
7	Successful HTTP connectivity from the Wazuh VM to n8n on the Windows host.	8
8	n8n Webhook node configured for POST requests on the <code>wazuh-alert</code> path with immediate response behavior.	9
9	n8n successfully receiving the manually generated Wazuh-like JSON alert at the test webhook.	10
10	Production webhook test sent from the Wazuh Manager host to n8n, returning a successful workflow-start response.	11
11	n8n execution showing the production webhook successfully receiving the Wazuh-side connectivity test payload.	11
12	Wazuh integration directory and Wazuh 4.14.6 server version verified before custom integration deployment.	12
13	Custom Wazuh integration files with ownership and executable permissions.	12
14	Manual validation of <code>custom-n8n</code> showing successful HTTP 200 delivery for the controlled alert.	14
15	n8n receiving the controlled alert forwarded by the custom Wazuh integration script.	14
16	Integration log showing automatic Wazuh alerts successfully forwarded to n8n with HTTP 200 responses.	15
17	n8n production execution containing a real Wazuh alert received through the custom integration.	16
18	Initial risk-based n8n workflow containing the Webhook, SOC Alert Triage, and notification decision nodes.	17
19	Medium-severity test event normalized to MEDIUM/P3 with analyst review and notification required.	18
20	Low-severity alert following the false branch with no SOC notification required.	19
21	Final n8n workflow: Webhook, SOC Alert Triage, risk decision, and email escalation.	19
22	Medium/P3 email notification generated by the n8n workflow for analyst review.	21
23	Successful n8n execution showing the complete workflow reaching the Send an Email node.	22
24	n8n execution history showing multiple successful workflow runs during final testing.	23
25	Critical alert processing result showing P1 classification and immediate SOC escalation logic.	24
26	Critical/P1 email escalation produced by the final workflow validation.	25

1 Executive Summary

This laboratory implemented a functional integration between Wazuh and n8n to automate the handling of security alerts. Wazuh acted as the security monitoring and detection platform, while n8n acted as the orchestration layer responsible for receiving alerts, normalizing relevant fields, assigning a risk-based priority, deciding whether analyst notification was required, and sending an email escalation when the configured threshold was met.

The implementation was built in stages so that each layer could be independently validated. First, the Wazuh Manager, network interfaces, Docker runtime, and n8n instance were verified. Second, a webhook endpoint in n8n was created and tested with a manually generated JSON payload. Third, a custom Wazuh integration named `custom-n8n` was created and validated independently. Fourth, the integration was connected to Wazuh through `ossec.conf`, allowing real Wazuh alerts to invoke the custom integration and reach n8n. Finally, an n8n workflow implemented risk-based triage and selective email escalation.

The unique automation use case was titled **Risk-Based Wazuh Alert Triage and Email Escalation using n8n**. Rather than forwarding every event as an urgent incident, the workflow used the Wazuh alert level to classify events into LOW/P4, MEDIUM/P3, HIGH/P2, and CRITICAL/P1 handling tiers. Alerts below the notification threshold followed a no-escalation branch, while higher-severity events triggered an analyst notification. This design is operationally useful because it reduces unnecessary alert noise while preserving human review before disruptive containment actions.

From a governance and security-management perspective, the implementation incorporated configuration backup, staged validation, separation of detection and orchestration duties, evidence collection, human-in-the-loop escalation, and exclusion of secrets from reusable artifacts. These choices align with common CISA, CISM, and CISSP principles around change control, auditability, incident response, least privilege, and risk-based decision-making.

2 Task Objectives and Deliverables

The task required the following outcomes:

1. Configure Wazuh and n8n integration using webhooks.
2. Verify that Wazuh security alerts are successfully received by n8n.
3. Design and implement one unique security automation use case.
4. Build a workflow that receives, processes, filters, acts on, and notifies on Wazuh alerts.
5. Test the workflow with multiple alert severities and validate execution.
6. Maintain evidence of successful alert delivery, processing, and notification.
7. Document architecture, webhook configuration, workflow logic, use case, results, findings, and recommendations.

The final deliverables achieved were a functional Wazuh-to-n8n integration, a custom automated alert-handling workflow, successful email notification, and screenshot evidence across the implementation lifecycle.

3 Integration Architecture

3.1 Logical Architecture

The implemented data path is shown below.

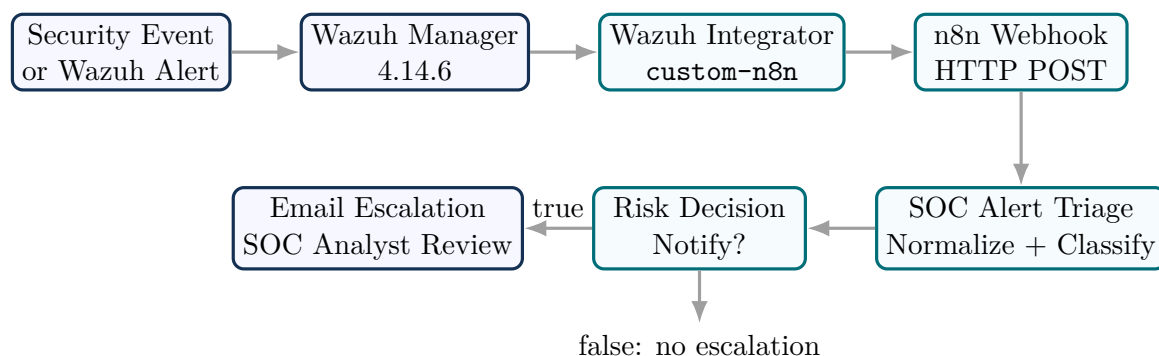


Figure 1: Logical architecture of the Wazuh-n8n alert automation workflow.

3.2 Network Placement

The Wazuh Manager ran inside a VMware Ubuntu virtual machine with address 192.168.149.128/24. n8n ran inside Docker on the Windows host and was reachable from the Wazuh VM through the VMware-facing Windows address 192.168.149.1 on TCP port 5678. The production webhook used by the Wazuh integration was therefore reachable at:

```
http://192.168.149.1:5678/webhook/wazuh-alert
```

This separation preserved a clear boundary between the SIEM/detection platform and the automation platform while still allowing direct lab connectivity.

4 Lab Environment and Baseline Validation

4.1 Wazuh Manager Status

The Wazuh Manager was first verified as active before any integration changes were made. This established a known-good baseline and reduced the chance of confusing pre-existing platform faults with integration problems.

```

suhaib@suhaib-VMware-Virtual-Platform:~/Desktop$ hostname -I
192.168.149.128
suhaib@suhaib-VMware-Virtual-Platform:~/Desktop$ sudo systemctl status wazuh-manager --no-pager
● wazuh-manager.service - Wazuh manager
   Loaded: loaded (/usr/lib/systemd/system/wazuh-manager.service; enabled; preset: enabled)
   Drop-In: /etc/systemd/system/wazuh-manager.service.d
           └─timeout.conf
   Active: active (running) since Sat 2026-08-15 12:55:42 PKT; 5min ago
   Invocation: 1b12f8db258140e3b3688f6e6d9c7542
   Process: 1390 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
   Tasks: 212 (limit: 8797)
   Memory: 1G (peak: 1G)
   CPU: 1min 59.400s
   CGroup: /system.slice/wazuh-manager.service
           └─2901 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
              2902 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
              2903 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
              2906 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
              2909 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
              2950 /var/ossec/bin/wazuh-authd
              2963 /var/ossec/bin/wazuh-db
              3000 /var/ossec/bin/wazuh-execd
              3012 /var/ossec/bin/wazuh-analysisd
              3023 /var/ossec/bin/wazuh-syscheckd
              3039 /var/ossec/bin/wazuh-remoted
              3077 /var/ossec/bin/wazuh-logcollector
              3097 /var/ossec/bin/wazuh-monitord
             3149 /var/ossec/bin/wazuh-modulesd

Aug 15 12:55:34 suhaib-VMware-Virtual-Platform env[1390]: Started wazuh-syscheckd...
Aug 15 12:55:36 suhaib-VMware-Virtual-Platform env[1390]: Started wazuh-remoted...
Aug 15 12:55:37 suhaib-VMware-Virtual-Platform env[1390]: Started wazuh-logcollector...
Aug 15 12:55:38 suhaib-VMware-Virtual-Platform env[1390]: Started wazuh-monitord...
Aug 15 12:55:38 suhaib-VMware-Virtual-Platform env[3146]: 2026/08/15 12:55:38 wazuh-modulesd:router: INFO: Loaded router module.
Aug 15 12:55:38 suhaib-VMware-Virtual-Platform env[3146]: 2026/08/15 12:55:38 wazuh-modulesd:content_manager: INFO: Loaded content_manager module.
Aug 15 12:55:38 suhaib-VMware-Virtual-Platform env[3146]: 2026/08/15 12:55:38 wazuh-modulesd:inventory-harvester: INFO: Loaded Inventory har...r module.
Aug 15 12:55:39 suhaib-VMware-Virtual-Platform env[1390]: Started wazuh-modulesd...
Aug 15 12:55:42 suhaib-VMware-Virtual-Platform env[1390]: Completed.
Aug 15 12:55:42 suhaib-VMware-Virtual-Platform systemd[1]: Started wazuh-manager.service - Wazuh manager.
Hint: Some lines were ellipsized, use -l to show in full.
suhaib@suhaib-VMware-Virtual-Platform:~/Desktop$

```

Figure 2: Wazuh Manager active and running on the Ubuntu virtual machine, with the host address visible for baseline validation.

4.2 Wazuh Network Interface

The Ubuntu VM network configuration showed interface `ens33` using `192.168.149.128/24`. This address was used to reason about the correct VMware network path to the Windows host.

```

suhaib@suhaib-VMware-Virtual-Platform:~/Desktop$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 00:0c:29:67:92:0c brd ff:ff:ff:ff:ff:ff
   inet 192.168.149.128/24 brd 192.168.149.255 scope global dynamic noprefixroute ens33
       valid_lft 1361sec preferred_lft 1361sec
   inet6 fe80::20c:29ff:fe67:920c/64 scope link noprefixroute
       valid_lft forever preferred_lft forever

```

Figure 3: Wazuh VM network configuration showing the active `ens33` interface and IPv4 address.

4.3 Docker Runtime Validation

Docker and Docker Compose were verified on the Windows host before n8n deployment. The environment used Docker 29.6.2 and Docker Compose v5.3.1.

```
PS C:\WINDOWS\system32> docker --version
Docker version 29.6.2, build dfc4efb
PS C:\WINDOWS\system32> docker compose version
Docker Compose version v5.3.1
```

Figure 4: Docker and Docker Compose versions verified on the Windows host.

5 n8n Deployment

5.1 Container Deployment

n8n was deployed as a Docker container with TCP port 5678 published to the Windows host. Persistent n8n state was stored in a Docker volume so that the workflow and configuration would survive container restarts.

A representative deployment command was:

```
docker run -d --name wazuh-n8n -p 5678:5678 \
-e TZ="Asia/Karachi" \
-e GENERIC_TIMEZONE="Asia/Karachi" \
-v n8n_data:/home/node/.n8n \
docker.n8n.io/n8nio/n8n
```

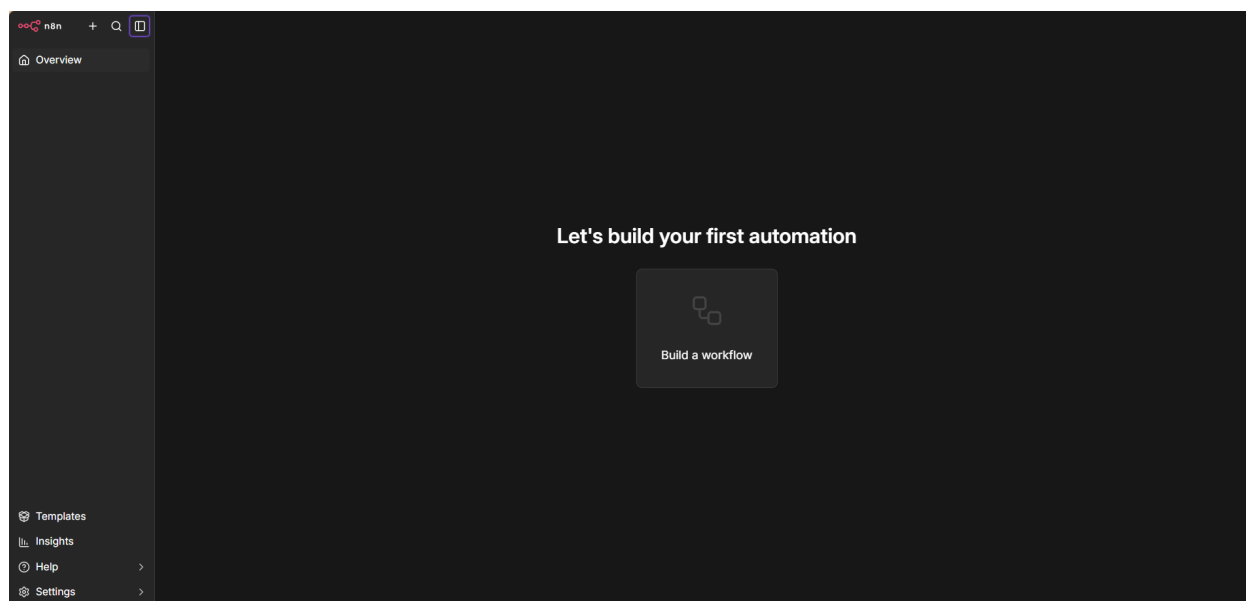


Figure 5: n8n web interface successfully running after deployment.

```
PS E:\wazuh-n8n-lab> docker ps
```

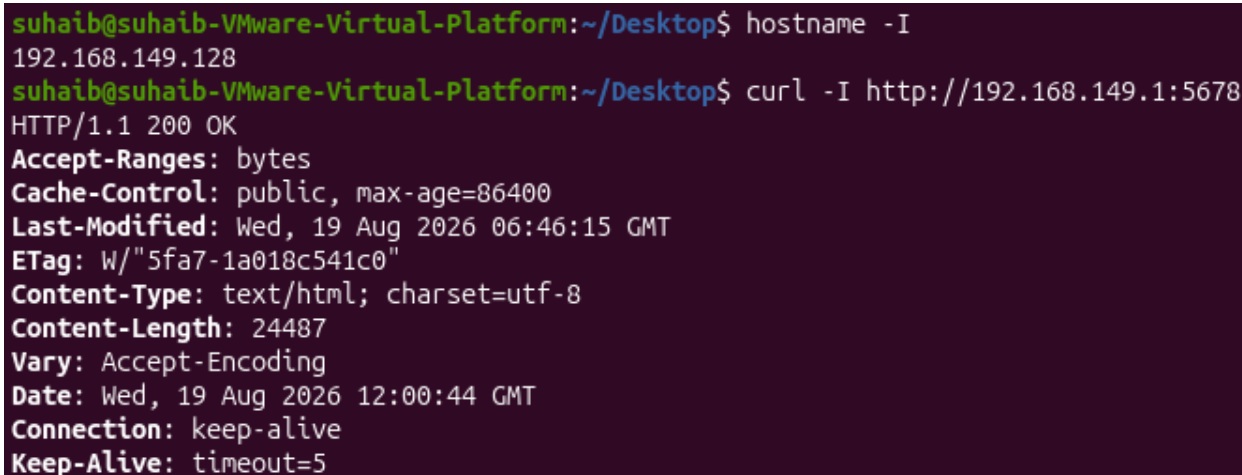
CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
67f15546f2f6	docker.n8n.io/n8nio/n8n	"tini -- /docker-ent..."	5 hours ago	Up 5 hours	0.0.0.0:5678->5678/tcp, [::]:5678->5678/tcp
	wazuh-n8n				

Figure 6: Docker container listing showing the n8n container and published port 5678.

5.2 Connectivity Validation from Wazuh to n8n

Before building the webhook integration, connectivity from the Wazuh VM to n8n was validated directly. The command below returned HTTP 200, proving that the VM could reach the Windows-hosted n8n service over the intended network path.

```
curl -I http://192.168.149.1:5678
```



```
suhaib@suhaib-VMware-Virtual-Platform:~/Desktop$ hostname -I
192.168.149.128
suhaib@suhaib-VMware-Virtual-Platform:~/Desktop$ curl -I http://192.168.149.1:5678
HTTP/1.1 200 OK
Accept-Ranges: bytes
Cache-Control: public, max-age=86400
Last-Modified: Wed, 19 Aug 2026 06:46:15 GMT
ETag: W/"5fa7-1a018c541c0"
Content-Type: text/html; charset=utf-8
Content-Length: 24487
Vary: Accept-Encoding
Date: Wed, 19 Aug 2026 12:00:44 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

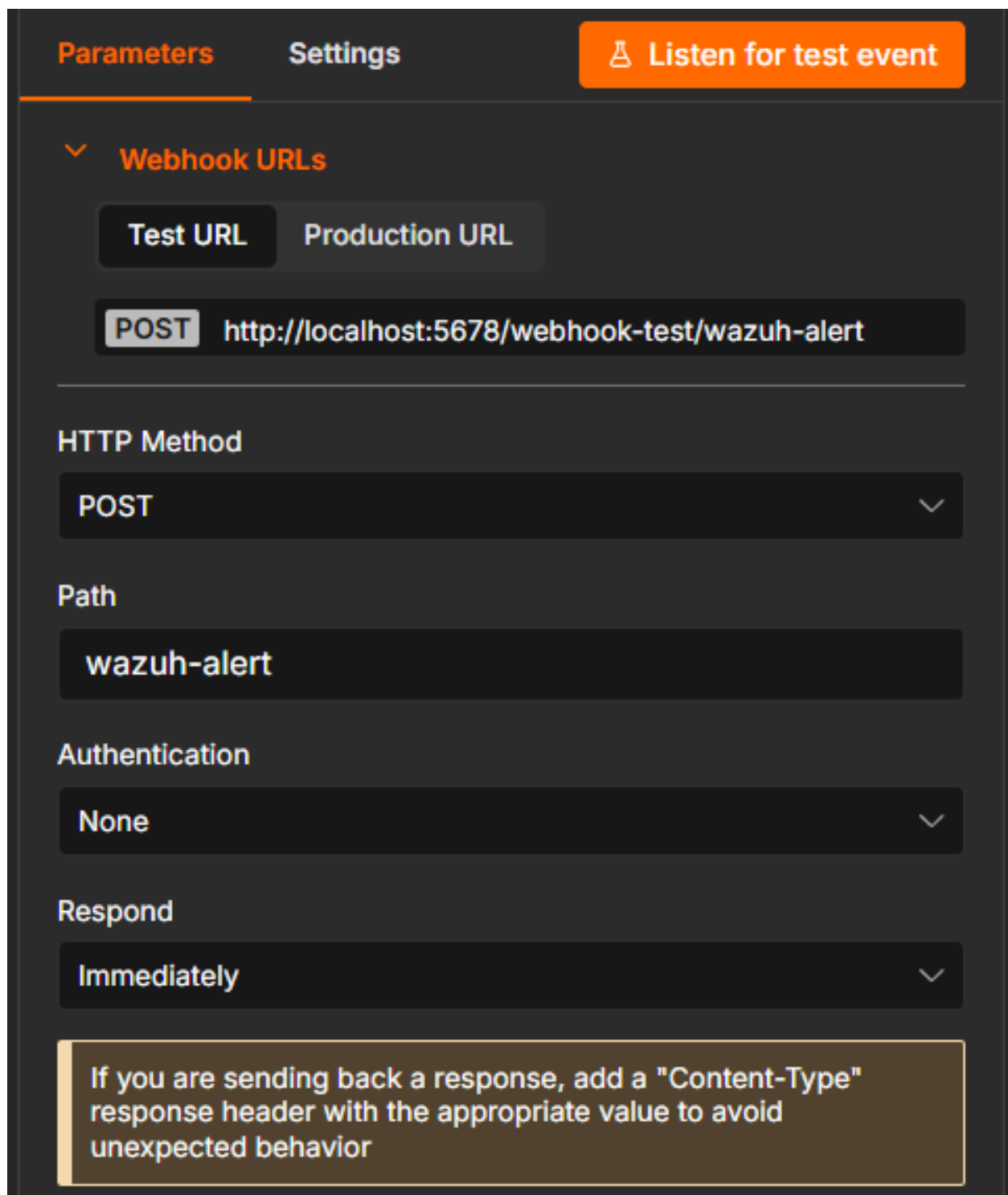
Figure 7: Successful HTTP connectivity from the Wazuh VM to n8n on the Windows host.

This staged connectivity test is significant from an audit and troubleshooting standpoint. It confirms that later integration failures would not be caused by a basic routing or service-reachability problem.

6 Webhook Configuration and Initial Validation

6.1 Webhook Design

An n8n Webhook node was configured with HTTP method POST and path `wazuh-alert`. During development, the test endpoint used the `/webhook-test/` path. After validation, the workflow was activated and Wazuh was configured to use the production `/webhook/` path.

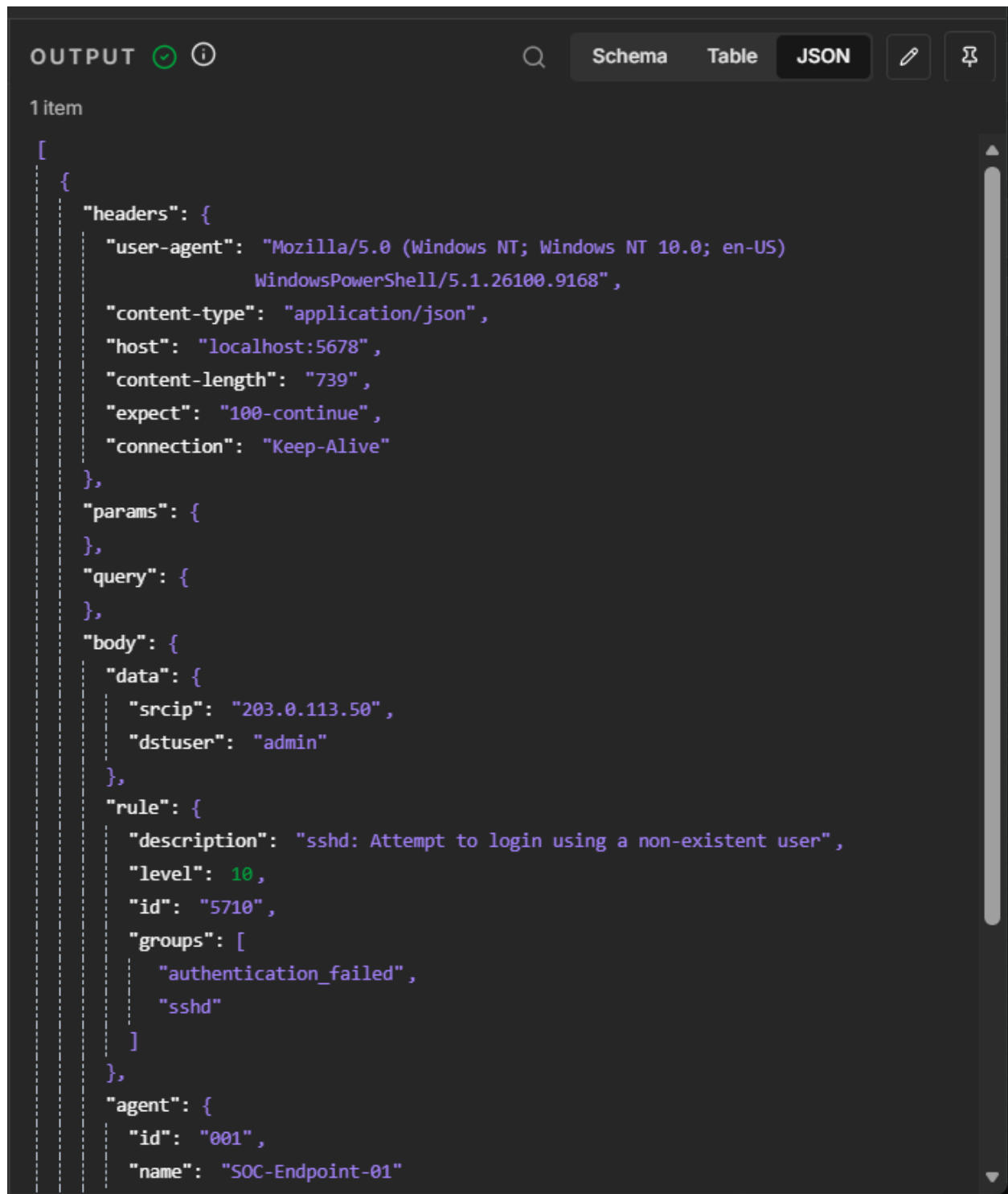


The screenshot shows the configuration interface for an n8n Webhook node. At the top, there are two tabs: 'Parameters' (selected) and 'Settings'. To the right of the tabs is an orange button with a flask icon and the text 'Listen for test event'. Below the tabs, there is a section titled 'Webhook URLs' with a dropdown arrow. Under this section, there are two buttons: 'Test URL' (selected) and 'Production URL'. Below these buttons, there is a text input field showing 'POST' in a grey box followed by the URL 'http://localhost:5678/webhook-test/wazuh-alert'. Below this, there is a section titled 'HTTP Method' with a dropdown menu showing 'POST'. Below that, there is a section titled 'Path' with a text input field showing 'wazuh-alert'. Below that, there is a section titled 'Authentication' with a dropdown menu showing 'None'. Below that, there is a section titled 'Respond' with a dropdown menu showing 'Immediately'. At the bottom, there is a yellow box with the text: 'If you are sending back a response, add a "Content-Type" response header with the appropriate value to avoid unexpected behavior'.

Figure 8: n8n Webhook node configured for POST requests on the `wazuh-alert` path with immediate response behavior.

6.2 Manual Test Event

A controlled Wazuh-like JSON event was sent to the test webhook. The payload included rule ID 5710, Wazuh level 10, agent context, a source IP from a documentation-only range, and a destination user. The purpose of this test was to validate the n8n application endpoint independently of the Wazuh Integrator.

The screenshot shows the n8n web interface with the 'OUTPUT' tab selected. It displays a single JSON item. The JSON structure is as follows:

```
[
  {
    "headers": {
      "user-agent": "Mozilla/5.0 (Windows NT; Windows NT 10.0; en-US) WindowsPowerShell/5.1.26100.9168",
      "content-type": "application/json",
      "host": "localhost:5678",
      "content-length": "739",
      "expect": "100-continue",
      "connection": "Keep-Alive"
    },
    "params": {},
    "query": {},
    "body": {
      "data": {
        "srcip": "203.0.113.50",
        "dstuser": "admin"
      },
      "rule": {
        "description": "sshd: Attempt to login using a non-existent user",
        "level": 10,
        "id": "5710",
        "groups": [
          "authentication_failed",
          "sshd"
        ]
      },
      "agent": {
        "id": "001",
        "name": "SOC-Endpoint-01"
      }
    }
  }
]
```

Figure 9: n8n successfully receiving the manually generated Wazuh-like JSON alert at the test webhook.

6.3 Production Webhook Test from the Wazuh VM

The workflow was then activated and the production URL was tested from the Wazuh VM using HTTP POST. The response `Workflow was started` confirmed that the production webhook was active and reachable.

```

suhaib@suhaib-VMware-Virtual-Platform:~/Desktop$ curl -X POST \
http://192.168.149.1:5678/webhook/wazuh-alert \
-H "Content-Type: application/json" \
-d '{
  "integration": "wazuh-connectivity-test",
  "timestamp": "2026-08-19T17:15:00+05:00",
  "rule": {
    "id": "100001",
    "level": 5,
    "description": "Wazuh to n8n production webhook connectivity test"
  },
  "agent": {
    "id": "000",
    "name": "wazuh-manager"
  }
}'
{"message":"Workflow was started"}suhaib@suhaib-VMware-Virtual-Platform:~/Desktop$

```

Figure 10: Production webhook test sent from the Wazuh Manager host to n8n, returning a successful workflow-start response.

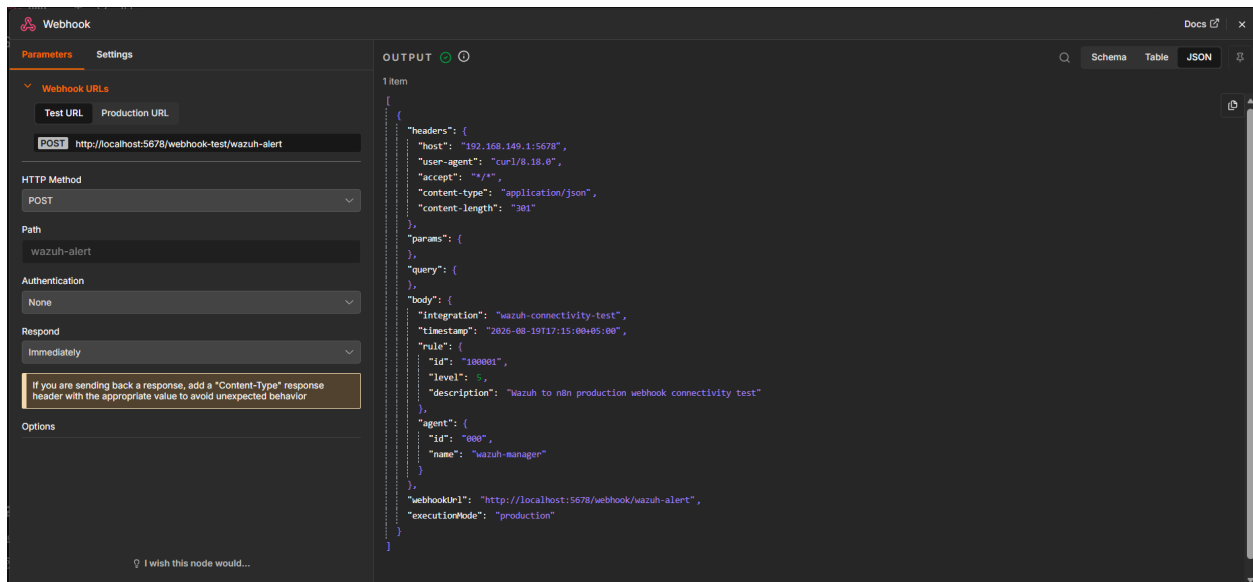


Figure 11: n8n execution showing the production webhook successfully receiving the Wazuh-side connectivity test payload.

7 Wazuh Custom Integration Implementation

7.1 Integration Environment

The Wazuh integration directory was inspected before adding custom files. The server reported Wazuh version v4.14.6, revision rc2, and no pre-existing custom-n8n integration. Existing integrations such as VirusTotal, Slack, Shuffle, PagerDuty, and Maltiverse were present, providing a reference pattern for launcher and Python helper files.

```

suhaib@suhaib-VMware-Virtual-Platform:~/Desktop$ sudo ls -la /var/ossec/integrations/
[sudo: authenticate] Password:
total 88
drwxr-x---  2 root wazuh  4096 Jul 12 02:44 .
drwxr-x--- 20 root wazuh  4096 Jul 12 02:46 ..
-rwxr-x---  1 root wazuh  1045 Jun 25 22:01 maltiverse
-rwxr-x---  1 root wazuh 20926 Jun 25 22:01 maltiverse.py
-rwxr-x---  1 root wazuh  1045 Jun 25 22:01 pagerduty
-rwxr-x---  1 root wazuh  6449 Jun 25 22:01 pagerduty.py
-rwxr-x---  1 root wazuh  1045 Jun 25 22:01 shuffle
-rwxr-x---  1 root wazuh  7249 Jun 25 22:01 shuffle.py
-rwxr-x---  1 root wazuh  1045 Jun 25 22:01 slack
-rwxr-x---  1 root wazuh  6835 Jun 25 22:01 slack.py
-rwxr-x---  1 root wazuh  1045 Jun 25 22:01 virustotal
-rwxr-x---  1 root wazuh 10691 Jun 25 22:01 virustotal.py
suhaib@suhaib-VMware-Virtual-Platform:~/Desktop$ sudo ls -la /var/ossec/integrations | grep custom
suhaib@suhaib-VMware-Virtual-Platform:~/Desktop$ sudo /var/ossec/bin/wazuh-control info
WAZUH_VERSION="v4.14.6"
WAZUH_REVISION="rc2"
WAZUH_TYPE="server"

```

Figure 12: Wazuh integration directory and Wazuh 4.14.6 server version verified before custom integration deployment.

7.2 Custom Integration Files

Two files were created:

- `/var/ossec/integrations/custom-n8n` - shell launcher compatible with the Wazuh integration mechanism.
- `/var/ossec/integrations/custom-n8n.py` - Python logic responsible for reading the JSON alert file and sending it to n8n via HTTP POST.

Both files were owned by `root:wazuh` and assigned executable permissions appropriate for the Wazuh service context.

```

root@suhaib-VMware-Virtual-Platform:~# sudo ls -l /var/ossec/integrations/custom-n8n*
-rwxr-x--- 1 root wazuh  373 Aug 19 23:22 /var/ossec/integrations/custom-n8n
-rwxr-x--- 1 root wazuh 2107 Aug 19 23:23 /var/ossec/integrations/custom-n8n.py
-rw-r--r-- 1 root root   373 Aug 19 22:47 /var/ossec/integrations/custom-n8nM

```

Figure 13: Custom Wazuh integration files with ownership and executable permissions.

7.3 Launcher Script

The shell launcher follows the same design used by built-in Wazuh integrations by invoking the Wazuh-bundled Python runtime and forwarding all arguments to the Python implementation.

```

#!/bin/sh

WPYTHON_BIN="framework/python/bin/python3"
SCRIPT_PATH_NAME="$0"
DIR_NAME="$(cd "$(dirname "${SCRIPT_PATH_NAME}")" && pwd -P)"
SCRIPT_NAME="$(basename "${SCRIPT_PATH_NAME}")"

if [ -z "${WAZUH_PATH}" ]; then
    WAZUH_PATH="$(cd "${DIR_NAME}/.." && pwd)"
fi

```

```
PYTHON_SCRIPT="${DIR_NAME}/${SCRIPT_NAME}.py"
"${WAZUH_PATH}/${WPYTHON_BIN}" "${PYTHON_SCRIPT}" "$@"
```

7.4 Python Forwarder Logic

The Python component performs the following operations:

1. Accepts the Wazuh alert file path as the first argument.
2. Reads and parses the alert as JSON.
3. Serializes the complete alert payload.
4. Sends the payload to the n8n production webhook with Content-Type: application/json.
5. Logs HTTP success/failure information to /var/ossec/logs/integrations.log.

The corrected core logic is represented below.

```
import json
import sys
import urllib.request
import urllib.error
from datetime import datetime, timezone

N8N_WEBHOOK_URL = "http://192.168.149.1:5678/webhook/wazuh-alert"
LOG_FILE = "/var/ossec/logs/integrations.log"

def log_message(message):
    timestamp = datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ")
    with open(LOG_FILE, "a", encoding="utf-8") as log:
        log.write(f"{timestamp} custom-n8n: {message}\n")

def main():
    if len(sys.argv) < 2:
        log_message(f"ERROR - No arguments received. Arguments: {sys.argv}")
        sys.exit(1)

    alert_file = sys.argv[1]

    try:
        with open(alert_file, "r", encoding="utf-8") as file:
            alert = json.load(file)
    except Exception as error:
        log_message(f"ERROR - Unable to read alert file '{alert_file}': {error}")
        sys.exit(1)

    payload = json.dumps(alert).encode("utf-8")
    request = urllib.request.Request(
        N8N_WEBHOOK_URL,
        data=payload,
        headers={
            "Content-Type": "application/json",
            "User-Agent": "Wazuh-custom-n8n"
        },
        method="POST"
    )

    try:
        with urllib.request.urlopen(request, timeout=10) as response:
```

```

rule = alert.get("rule", {})
agent = alert.get("agent", {})
log_message(
    "SUCCESS - "
    f"HTTP {response.getcode()} | "
    f"Rule ID: {rule.get('id', 'N/A')} | "
    f"Level: {rule.get('level', 'N/A')} | "
    f"Agent: {agent.get('name', 'manager'))}"
)
except Exception as error:
    log_message(f"ERROR - Failed to send alert to n8n: {error}")
    sys.exit(1)

```

7.5 Manual Integration Validation

Before connecting the custom integration to the Wazuh service, a controlled alert file was passed directly to the script. The integration log recorded HTTP 200 for rule ID 100100 at level 10, confirming that the launcher, Python logic, network path, and n8n production endpoint worked together.

```

suhaib@suhaib-VMware-Virtual-Platform:~$ sudo /var/ossec/integrations/custom-n8n ~/n8n-test-alert.json
suhaib@suhaib-VMware-Virtual-Platform:~$ sudo tail -n 10 /var/ossec/logs/integrations.log
2026-08-19T18:30:28Z custom-n8n: SUCCESS - HTTP 200 | Rule ID: 100100 | Level: 10 | Agent: wazuh-manager
2026-08-19T18:39:06Z custom-n8n: SUCCESS - HTTP 200 | Rule ID: 100100 | Level: 10 | Agent: wazuh-manager

```

Figure 14: Manual validation of custom-n8n showing successful HTTP 200 delivery for the controlled alert.

OUTPUT					
1 item					
headers	params	query	body	webhookUrl	executionMode
accept-encoding : Identity content-length : 394 host : 192.168.149.1:5678 content-type : application/json user-agent : Wazuh-custom-n8n connection : close	{empty object}	{empty object}	timestamp : 2026-08-19T17:30:00+05:00 rule level : 10 description : Controlled high-risk SSH authentication failure for Wazuh-n8n integration testing id : 100100 groups 0 : authentication_failed 1 : sshd agent id : 000 name : wazuh-manager manager name : wazuh-manager data srcip : 203.0.113.50 dstuser : admin integration_test : true	http://localhost:5678/webhook/wazuh-alert	production

Figure 15: n8n receiving the controlled alert forwarded by the custom Wazuh integration script.

8 Wazuh Integrator Configuration

8.1 Change Control and Backup

Before modifying the Wazuh configuration, a backup copy of `/var/ossec/etc/ossec.conf` was created. This provided a rollback path if the new integration caused the manager to fail validation or restart.

The added configuration block was:

```
<ossec_config>
  <integration>
    <name>custom-n8n</name>
    <level>5</level>
    <alert_format>json</alert_format>
  </integration>
</ossec_config>
```

The `level` value of 5 means Wazuh can forward events at level 5 and above to n8n, while the n8n workflow performs the finer-grained triage and escalation decision.

After modification, the Wazuh analysis configuration was validated using:

```
sudo /var/ossec/bin/wazuh-analysisd -t
```

No validation error was returned, after which the Wazuh Manager service was restarted.

8.2 Troubleshooting the Wazuh Argument Format

During the first automatic invocation, the integration log showed that the Python script attempted to open `> /dev/null 2&1` as the alert file. The initial implementation used `sys.argv[-1]`, assuming the alert path would always be the final argument. Wazuh passed additional arguments, so the last item was not the alert file.

The issue was diagnosed using `/var/ossec/logs/integrations.log` and the manager process output. The script was corrected to use:

```
alert_file = sys.argv[1]
```

After this change, both controlled and automatically generated Wazuh alerts were successfully delivered. This troubleshooting step demonstrates the value of staged testing and integration-specific logging.

9 Real Wazuh-to-n8n Alert Delivery

Following the configuration change and script correction, Wazuh invoked `custom-n8n` automatically for actual alerts. The integration log recorded multiple HTTP 200 deliveries, including real Wazuh rule ID 533, level 7, generated by the Wazuh Manager host.

```
2026-08-19T21:17:19Z custom-n8n: SUCCESS - HTTP 200 | Rule ID: 533 | Level: 7 | Agent: suhaib-VMware-Virtual-Platform
2026-08-19T21:35:21Z custom-n8n: SUCCESS - HTTP 200 | Rule ID: 533 | Level: 7 | Agent: suhaib-VMware-Virtual-Platform
2026-08-19T21:41:20Z custom-n8n: SUCCESS - HTTP 200 | Rule ID: 533 | Level: 7 | Agent: suhaib-VMware-Virtual-Platform
2026-08-19T21:49:14Z custom-n8n: SUCCESS - HTTP 200 | Rule ID: 100100 | Level: 10 | Agent: wazuh-manager
```

Figure 16: Integration log showing automatic Wazuh alerts successfully forwarded to n8n with HTTP 200 responses.

OUTPUT 🔍 Schema Table JSON

1 item

headers	params	query	body	webhookUri	executionMode
accept-encoding : identity content-length : 708 host : 192.168.149.1:5678 content-type : application/json user-agent : Wazuh-custom-n8n connection : close	{empty object}	{empty object}	timestamp : 2026-08-20T03:03:08.177+0500 rule level : 7 description : Host-based anomaly detection event (rootcheck). id : 510 firedtimes : 20 mail : false groups 0 : ossec 1 : rootcheck pci_dss 0 : 10.6.1 gdpr 0 : IV_35.7.d agent id : 000 name : suhaib-VMware-Virtual-Platform manager name : suhaib-VMware-Virtual-Platform id : 1787176988.48300 full_log : Trojaned version of file '/usr/bin/md5sum' detected. Signature used: 'bash ^/bin/sh file h proc h dev [*cfhlnrstuw] ^/bin/.*/sh' (Generic). decoder name : rootcheck data title : Trojaned version of file detected.	http://localhost:5678/webhook/wazuh-alert	production

Figure 17: n8n production execution containing a real Wazuh alert received through the custom integration.

At this point, the core integration objective was achieved: a real Wazuh alert could traverse the Wazuh Integrator, custom forwarder, network path, and n8n production webhook without manual intervention.

10 Unique Automation Use Case

10.1 Use Case Definition

The implemented use case is **Risk-Based Wazuh Alert Triage and Email Escalation**. The goal is to avoid treating every Wazuh event as equally urgent. Instead, n8n normalizes the incoming event, maps the Wazuh alert level to an operational priority, and escalates only alerts meeting the notification threshold.

This use case is suitable for a SOC environment because it separates three responsibilities:

- **Detection:** Wazuh decides that an event meets a detection rule.
- **Orchestration:** n8n normalizes the event and determines the response path.
- **Human validation:** an analyst receives the notification and decides whether investigation or containment is warranted.

No automatic IP blocking or host isolation was configured. This was intentional: a severity score alone does not prove malicious intent, and automatic disruptive containment can create availability risk or block legitimate users. The workflow therefore automates triage and escalation while retaining analyst approval for consequential response actions.

11 n8n Workflow Design and Logic

11.1 Workflow Nodes

The final workflow contains four primary nodes:

1. **Webhook** - receives the Wazuh JSON event using HTTP POST.
2. **SOC Alert Triage** - extracts core fields, assigns severity/priority, and sets a notification flag.
3. **Requires SOC Notification?** - routes events based on `requires_notification`.
4. **Send an Email** - sends a formatted SOC alert when the true branch is selected.

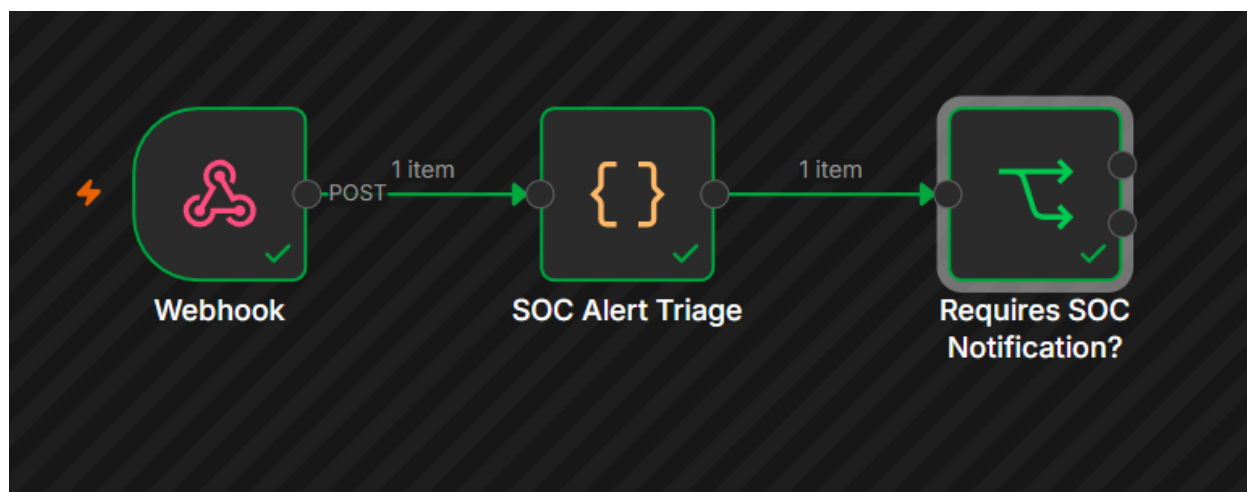


Figure 18: Initial risk-based n8n workflow containing the Webhook, SOC Alert Triage, and notification decision nodes.

11.2 Alert Normalization and Classification

The Code node used the incoming Wazuh object to derive normalized fields such as rule ID, Wazuh level, description, agent, source IP, destination user, severity, priority, automated action, and whether notification was required.

The implemented JavaScript logic was:

```
const alert = $json.body ?? $json;

const rule = alert.rule ?? {};
const agent = alert.agent ?? {};
const data = alert.data ?? {};
const level = Number(rule.level ?? 0);

let severity = "LOW";
let action = "LOG_ONLY";
let priority = "P4";

if (level >= 12) {
  severity = "CRITICAL";
  action = "IMMEDIATE_SOC_ESCALATION";
  priority = "P1";
} else if (level >= 10) {
  severity = "HIGH";
  action = "SOC_ESCALATION";
  priority = "P2";
} else if (level >= 7) {
  severity = "MEDIUM";
```

```

    action = "ANALYST_REVIEW";
    priority = "P3";
}

return {
  json: {
    timestamp: alert.timestamp ?? new Date().toISOString(),
    rule_id: rule.id ?? "N/A",
    rule_level: level,
    description: rule.description ?? "No description",
    severity,
    priority,
    automated_action: action,
    agent_id: agent.id ?? "N/A",
    agent_name: agent.name ?? "Wazuh Manager",
    source_ip: data.srcip ?? data.src_ip ?? "N/A",
    destination_user: data.dstuser ?? data.user ?? "N/A",
    manager: alert.manager?.name ?? "wazuh-manager",
    requires_notification: level >= 7,
    original_event: alert
  }
};

```

The resulting decision model is shown in Table 1.

Table 1: Risk-based alert triage model implemented in n8n.

Wazuh Level	Severity	Priority	Action	Notify
0-6	LOW	P4	LOG_ONLY	No
7-9	MEDIUM	P3	ANALYST_REVIEW	Yes
10-11	HIGH	P2	SOC_ESCALATION	Yes
12+	CRITICAL	P1	IMMEDIATE_SOC_ESCALATION	Yes

The screenshot displays the n8n workflow editor for a node titled "Requires SOC Notification?". The left pane shows the "INPUT" JSON, which is expanded to reveal details of a Wazuh alert, including its timestamp, rule ID (533), rule level (7), description, severity (MEDIUM), priority (P3), and automated action (ANALYST_REVIEW). The center pane shows the "Parameters" tab with a condition set to "{{\$json.requires_notification}}" and a dropdown set to "is true". The right pane shows the "OUTPUT" table, which contains one item with the following data: timestamp "2026-08-22T12:10:00+05:00", rule_id "533", rule_level "7", description "Manual test alert for SOC triage validation", and severity "MEDIUM".

Figure 19: Medium-severity test event normalized to MEDIUM/P3 with analyst review and notification required.

11.3 False Branch Validation

A low-severity level 5 event was used to prove that the automation does not escalate every event. The workflow classified the event as LOW/P4 with LOG_ONLY and `requires_notification=false`. The IF node routed the event through the false branch.

The screenshot displays the n8n workflow editor for a node titled "Requires SOC Notification?". The left pane shows the input data for a "SOC Alert Triage" event. The right pane shows the output for the "False Branch (1 item)".

INPUT Data:

- timestamp: 2026-08-22T12:15:00+05:00
- rule_id: 100200
- rule_level: 5
- description: Low severity test event
- severity: LOW
- priority: P4
- automated_action: LOG_ONLY
- agent_id: 000
- agent_name: wazuh-manager
- source_ip: 203.0.113.60
- destination_user: testuser
- manager: wazuh-manager
- requires_notification: false
- original_event:
 - rule:
 - description: Low severity test event
 - level: 5
 - id: 100200
 - timestamp: 2026-08-22T12:15:00+05:00
 - data: [redacted]

Conditions: `{{ $json.requires_notification }}` Is true

OUTPUT Data (False Branch):

timestamp	rule_id	rule_level	description	severity
2026-08-22T12:15:00+05:00	100200	5	Low severity test event	LOW

Figure 20: Low-severity alert following the false branch with no SOC notification required.

11.4 Final Workflow

The completed workflow connects the true branch of the notification decision to the email action.

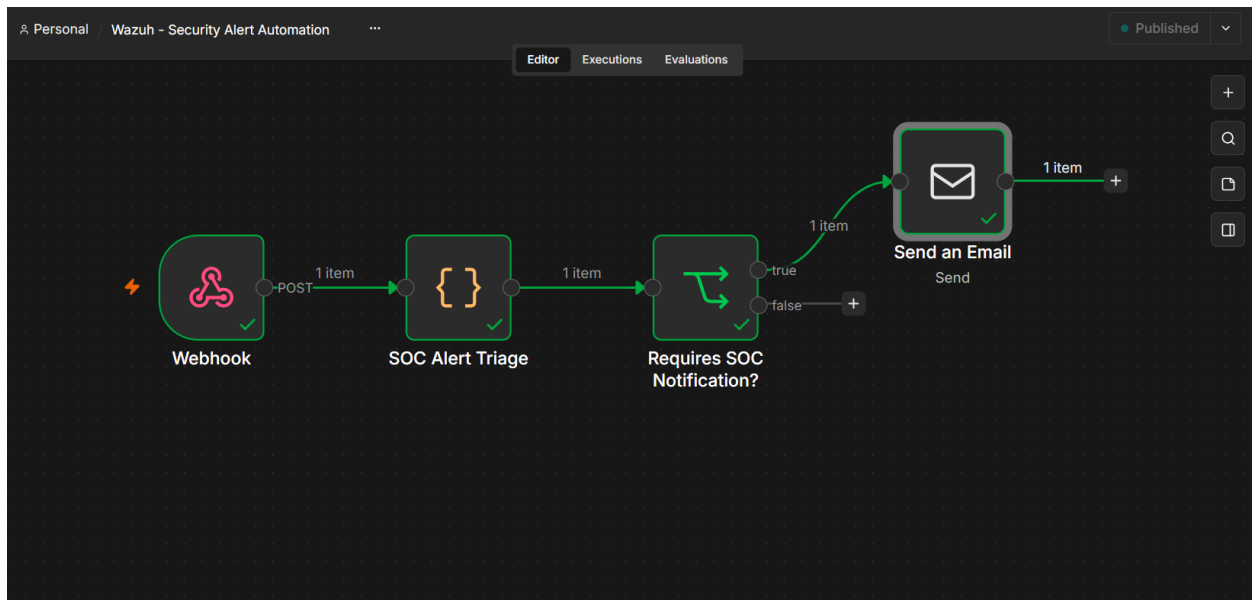


Figure 21: Final n8n workflow: Webhook, SOC Alert Triage, risk decision, and email escalation.

12 Email Notification Automation

12.1 Notification Content

Email was selected as the communication platform because it provided rapid implementation, clear audit evidence, and a channel that is familiar in both SOC and governance contexts. Credentials were configured inside n8n and were not included in screenshots, source code, or repository artifacts.

The notification body included:

- Severity and priority.
- Wazuh rule ID and rule level.
- Alert description.
- Affected agent.
- Source IP and destination user when present.
- Automated triage action.
- Status indicating pending SOC analyst review.

The phrase *Pending SOC Analyst Review* was intentionally retained to avoid treating an automated detection as a confirmed incident before human validation.

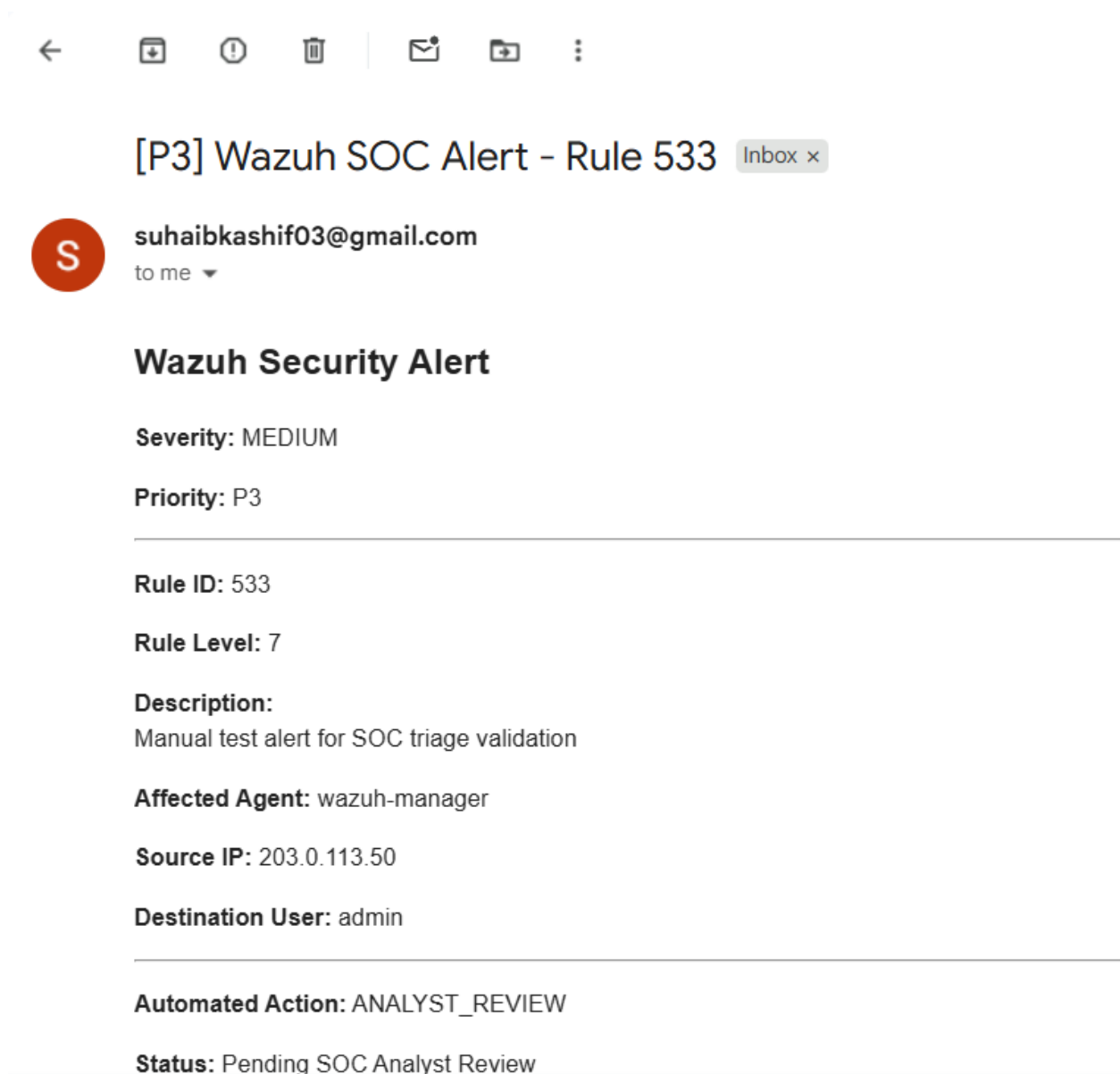


Figure 22: Medium/P3 email notification generated by the n8n workflow for analyst review.

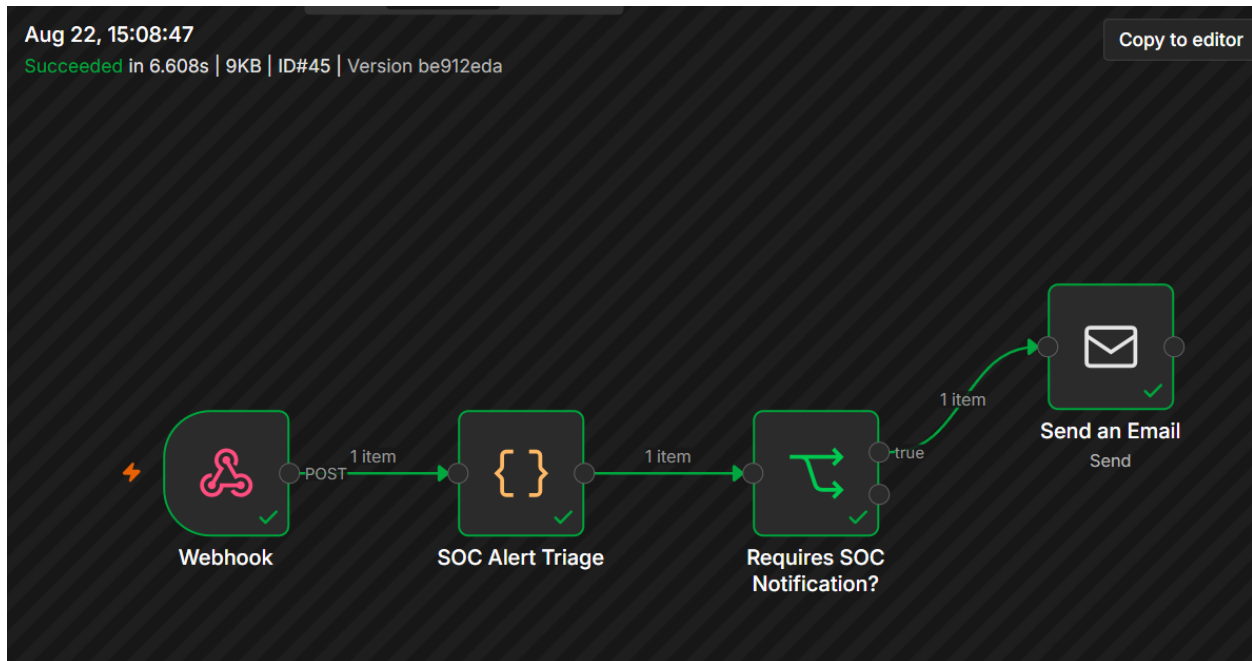


Figure 23: Successful n8n execution showing the complete workflow reaching the Send an Email node.

13 Multi-Severity Workflow Testing

13.1 Testing Method

The full integration path had already been proven with real Wazuh alerts. To test the n8n decision logic quickly and repeatably, controlled HTTP POST payloads were generated from PowerShell at multiple Wazuh alert levels. This separation is important: real alerts validated the transport/integration chain, while synthetic controlled alerts validated the orchestration branches and expected classification behavior.

Documentation-range addresses in 203.0.113.0/24 were used in the controlled tests so that the lab did not represent arbitrary real-world IP addresses as malicious.

13.2 Test Cases and Expected Results

Table 2: Final multi-severity test matrix.

Test	Level	Severity	Priority	Action	Email
Low	5	LOW	P4	LOG_ONLY	No
Medium	7	MEDIUM	P3	ANALYST_REVIEW	Yes
High	10	HIGH	P2	SOC_ESCALATION	Yes
Critical	12	CRITICAL	P1	IMMEDIATE_SOC_ESCALATION	Yes

13.3 Execution Evidence

Multiple workflow executions completed successfully during the final validation stage.

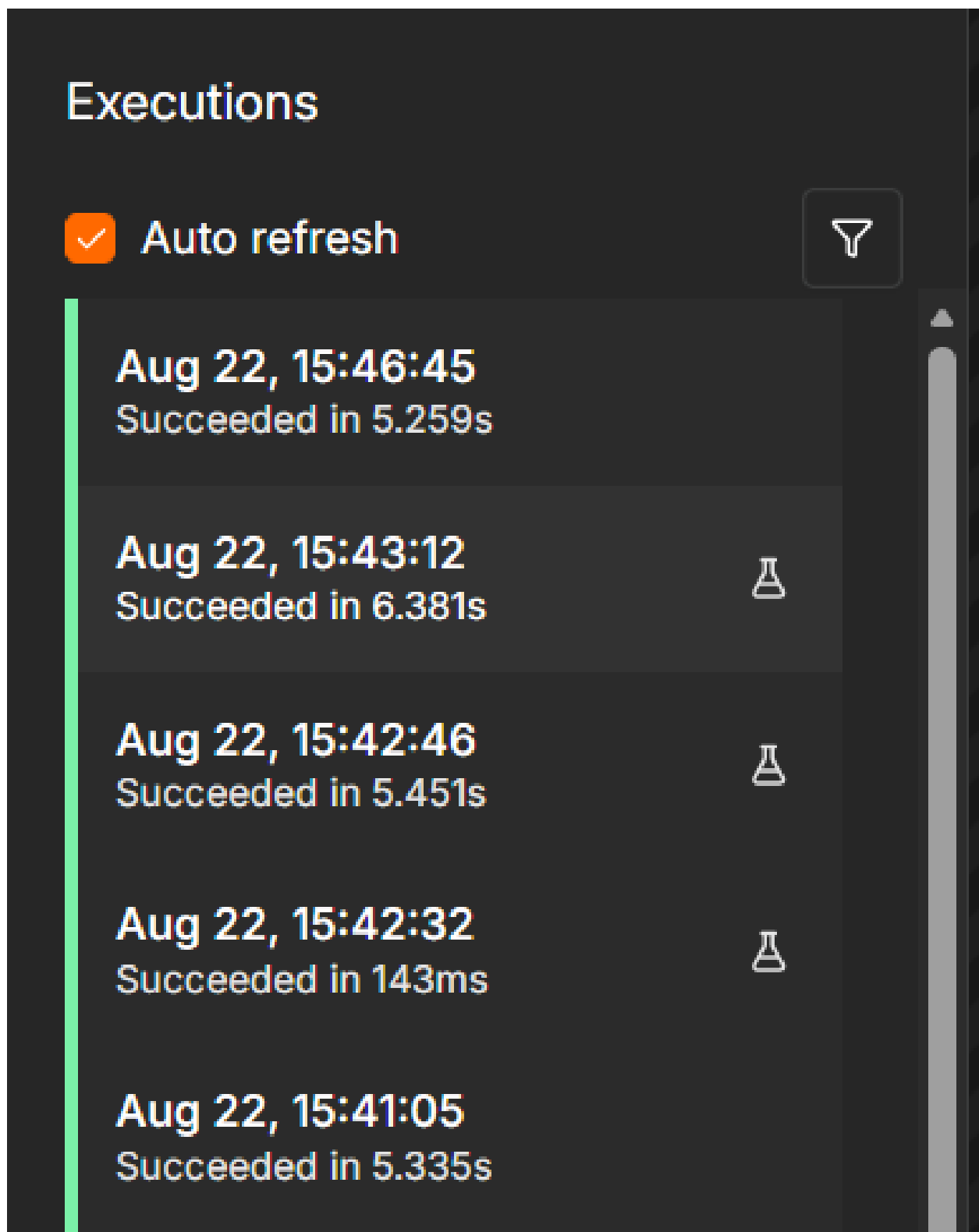


Figure 24: n8n execution history showing multiple successful workflow runs during final testing.

The critical test was processed as level 12, CRITICAL, P1, with IMMEDIATE_SOC_ESCALATION and notification required.

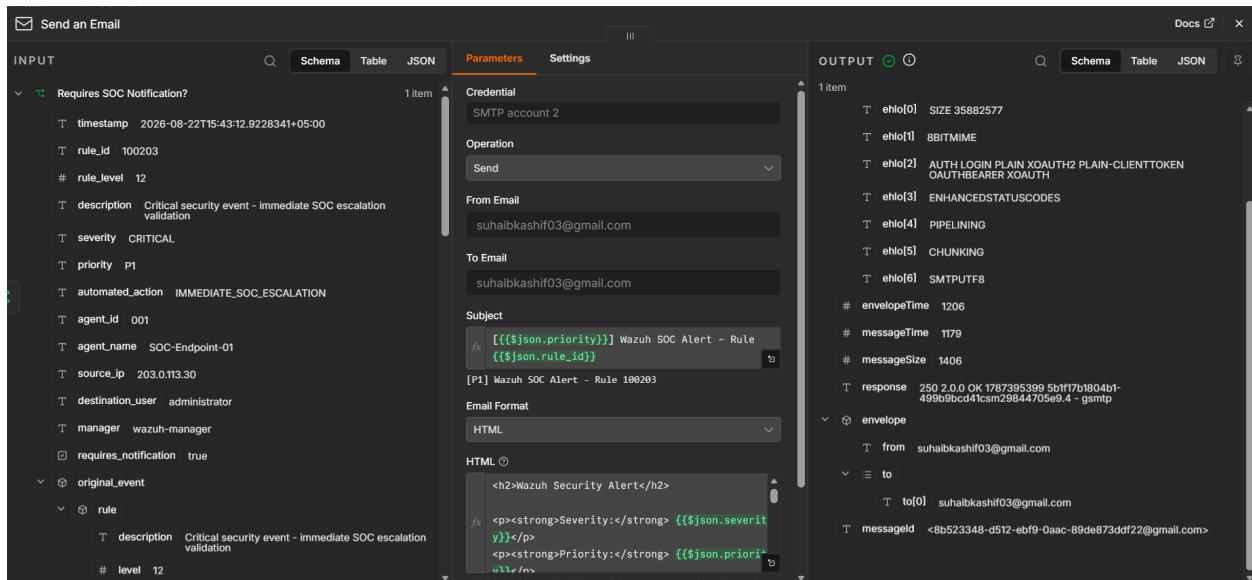


Figure 25: Critical alert processing result showing P1 classification and immediate SOC escalation logic.

The resulting email preserved the same triage information and clearly identified the alert as CRITICAL/P1.

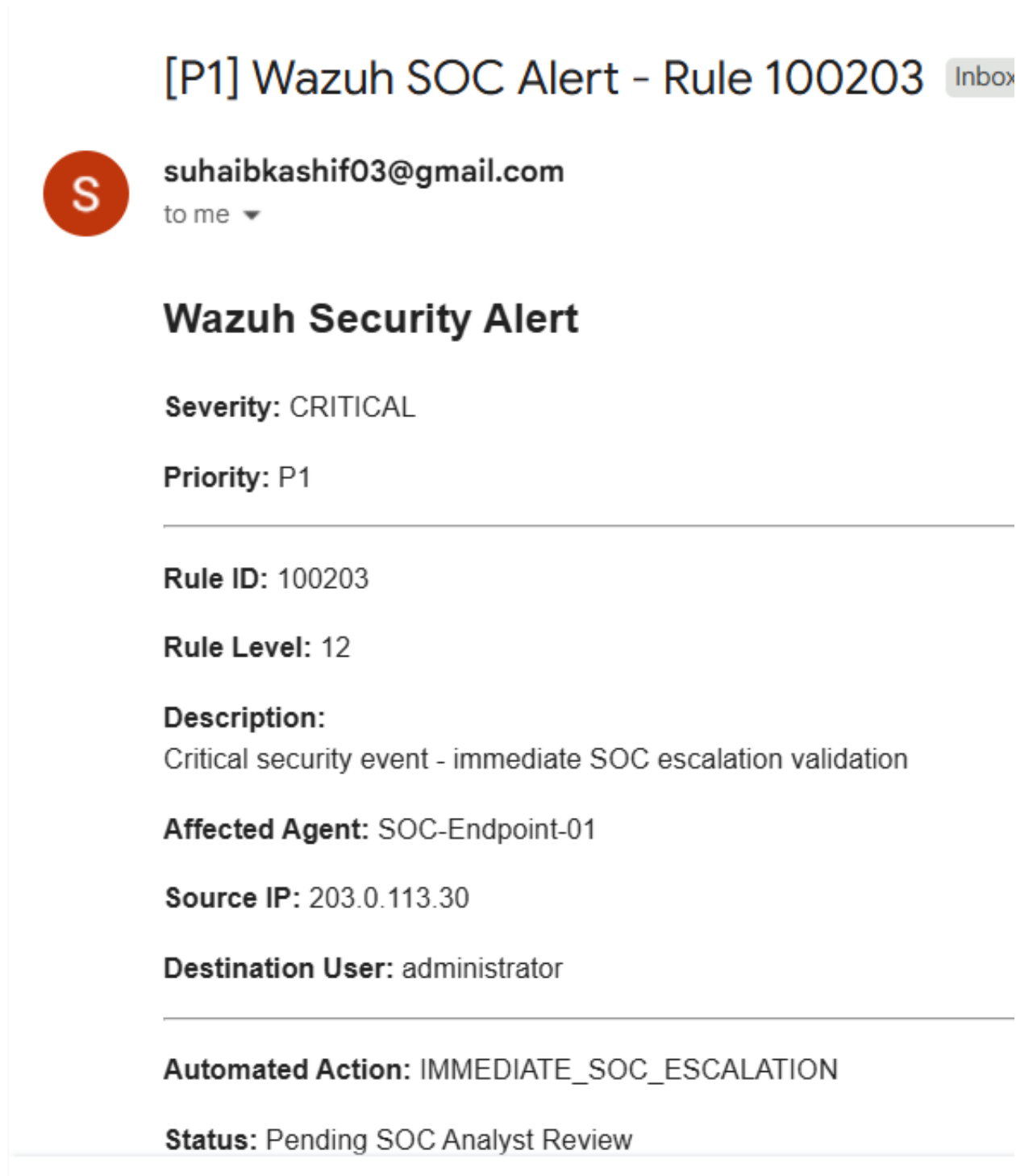


Figure 26: Critical/P1 email escalation produced by the final workflow validation.

14 Results

The implementation satisfied the functional requirements of Task 8. The key results were:

- Wazuh Manager and n8n infrastructure were independently validated before integration changes.
- n8n test and production webhook endpoints successfully received JSON security alerts.
- A custom Wazuh integration was created using a shell launcher and Python forwarder.
- Wazuh automatically invoked the custom integration for real alerts.

- Integration logs recorded HTTP 200 delivery to n8n.
- n8n normalized Wazuh alert data and assigned severity, priority, action, and notification state.
- A low-severity alert was correctly suppressed from email escalation.
- Medium, high, and critical test alerts successfully followed the notification branch.
- Email notifications contained sufficient context for analyst review without exposing credentials.

15 SOC, CISA, CISM, and CISSP Considerations

15.1 SOC Operations Perspective

The workflow supports a common SOC process: detection, triage, prioritization, escalation, and human review. Wazuh remains responsible for detection, while n8n performs deterministic orchestration. This allows the SOC to adjust escalation rules without changing detection content and provides a clear place to add future enrichment or ticket creation.

15.2 CISA-Oriented Control and Audit Perspective

From an audit/control perspective, the implementation included several verifiable practices:

- **Configuration backup:** `ossec.conf` was backed up before change.
- **Pre-change validation:** infrastructure and connectivity were tested independently.
- **Post-change validation:** Wazuh configuration was syntax-tested before restart.
- **Evidence retention:** screenshots and integration logs provide an audit trail of successful control operation.
- **Exception visibility:** integration errors were visible in `integrations.log`, enabling troubleshooting and control-failure detection.

15.3 CISM-Oriented Risk and Incident Management Perspective

The workflow applies risk-based escalation rather than uniform notification. The classification model is deterministic and transparent, which makes it easier to review, tune, and explain to stakeholders. The escalation email explicitly indicates pending analyst review, supporting an incident-management process in which automated detection is separated from incident confirmation.

15.4 CISSP-Oriented Security Architecture Perspective

The implementation reflects several security architecture principles:

- Separation of monitoring, orchestration, and human-response functions.
- Avoidance of automatic disruptive containment without sufficient validation.
- Explicit handling of secrets outside the source code and screenshots.
- Use of a dedicated production webhook path after test validation.
- Least-functionality mindset: only the automation required for the task was enabled.

16 Findings

1. **The integration path is reliable when each layer is tested independently.** Validating Docker, TCP reachability, test webhook behavior, production webhook behavior, and the custom integration separately made troubleshooting significantly easier.
2. **Wazuh custom integrations require careful argument handling.** The initial assumption that the alert file was the final process argument was incorrect in this environment. Integration logs exposed the problem and led to the correct use of `sys.argv[1]`.
3. **Risk-based filtering reduces notification noise.** The level 5 test followed the non-notification branch, proving that the workflow does not simply convert every Wazuh event into an email.
4. **Human-in-the-loop response is appropriate for this lab.** Severity-based alerting is useful for prioritization, but it should not by itself trigger irreversible containment actions.
5. **Integration logs are operational evidence.** HTTP 200 entries with rule ID, level, and agent provided direct evidence that the forwarder was functioning.

17 Recommendations

For a production implementation, the following improvements are recommended:

1. **Secure the webhook.** Add authentication or a shared secret and place n8n behind TLS/reverse proxy protection rather than using unauthenticated HTTP on a lab network.
2. **Restrict network exposure.** Limit TCP 5678 access to approved source addresses or expose n8n only through a hardened reverse proxy.
3. **Protect credentials.** Keep SMTP credentials exclusively in the n8n credential store or an external secrets manager; never commit them to Git.
4. **Add deduplication and rate limiting.** High-volume alerts should be grouped or rate-limited to reduce email flooding.
5. **Add enrichment.** Future workflows could enrich source IPs, hashes, users, or hosts using approved threat-intelligence sources before escalation.
6. **Add case management.** High and critical events could automatically create a ticket in an approved incident-management platform while preserving human approval for containment.
7. **Use context beyond Wazuh level.** Production prioritization should consider asset criticality, threat intelligence, authentication context, business impact, and repeated-event frequency rather than relying only on severity level.
8. **Monitor integration health.** Create a separate health check or alert when the custom integration cannot reach n8n, so automation failure itself becomes visible.
9. **Document change ownership.** Production changes should be formally approved, version-controlled, and linked to rollback instructions.

18 Conclusion

Task 8 successfully demonstrated an end-to-end Wazuh and n8n security automation workflow. Real Wazuh alerts were forwarded through a custom integration into an active n8n production webhook. n8n then transformed the alert into a normalized SOC record, applied deterministic risk-based triage,

routed low-severity events away from escalation, and sent email notifications for alerts requiring analyst attention.

The final result is more than a simple webhook demonstration: it is a compact SOAR-style workflow that shows integration engineering, alert triage, selective escalation, operational troubleshooting, evidence collection, and security-governance awareness. The design can be extended in future work with enrichment, ticketing, approval workflows, deduplication, and carefully controlled response actions.

A Evidence Index

No.	File	Evidence Purpose
01	01-wazuh-manager-status.png	Wazuh Manager baseline status and IP.
02	02-wazuh-network-configuration.png	Wazuh VM network interface and IPv4 address.
03	03-docker-environment.png	Docker and Compose prerequisites.
04	04-n8n-running.png	n8n UI availability.
05	05-n8n-docker-container.png	n8n container and published port.
06	06-wazuh-to-n8n-connectivity.png	HTTP reachability from Wazuh VM to n8n.
07	07-n8n-webhook-received-alert.png	Test webhook receiving JSON alert.
08	08-n8n-webhook-configuration.png	Webhook POST path configuration.
09	09-wazuh-to-n8n-production-webhook.png	Production webhook test from Wazuh host.
10	10-n8n-production-alert-received.png	Production execution received in n8n.
11	11-wazuh-custom-integration-environment.png	Wazuh version and integration directory.
12	12-custom-n8n-integration-files.png	Custom integration files and permissions.
13	13-custom-n8n-successful-delivery.png	Manual custom integration HTTP 200 success.
14	14-n8n-received-custom-wazuh-alert.png	n8n receiving controlled alert from custom integration.
15	15-real-wazuh-integration-success.png	Automatic real Wazuh integration delivery log.
16	16-real-wazuh-alert-received-n8n.png	Real Wazuh alert in n8n production execution.
17	17-risk-based-soc-workflow.png	Risk-based workflow structure.
18	18-alert-risk-classification.png	MEDIUM/P3 triage result.
19	19-low-severity-no-escalation.png	LOW/P4 false branch/no notification.
20	20-complete-soc-automation-workflow.png	Complete workflow including email action.
21	21-soc-email-notification.png	Medium/P3 email notification.
22	22-successful-email-notification-execution.png	Successful end-to-end notification execution.
23	23-multiple-alert-workflow-executions.png	Multiple successful test executions.
24	24-critical-alert-processing-result.png	CRITICAL/P1 triage and escalation result.
25	25-high-critical-email-notification.png	Critical email escalation evidence.

B Repository-Ready Artifacts

For a public GitHub portfolio repository, only sanitized and reusable artifacts should be committed. A suitable structure is:

```
wazuh-n8n-security-automation/  
|-- README.md  
|-- architecture/  
|-- wazuh/  
|   |-- integrations/  
|   |   |-- custom-n8n  
|   |   '-- custom-n8n.py  
|   '-- config/  
|       '-- integration-config.xml  
|-- n8n/  
|   '-- workflow/  
|       '-- wazuh-security-alert-automation.json  
|-- evidence/  
|-- docs/  
 '-- report/
```

Before publication, the n8n workflow export and all source files should be reviewed for credentials, email addresses, tokens, internal-only URLs, and secrets. The GitHub repository screenshot was not required to validate the core Task 8 deliverables and is therefore not used as evidence in this report.

C Reference Configuration Snippets

C.1 Wazuh Integration Block

```
<ossec_config>  
  <integration>  
    <name>custom-n8n</name>  
    <level>5</level>  
    <alert_format>json</alert_format>  
  </integration>  
</ossec_config>
```

C.2 Example Controlled Alert Structure

```
{  
  "timestamp": "2026-08-22T12:10:00+05:00",  
  "rule": {  
    "id": "533",  
    "level": 7,  
    "description": "Manual test alert for SOC triage validation"  
  },  
  "agent": {  
    "id": "000",  
    "name": "wazuh-manager"  
  },  
  "data": {  
    "srcip": "203.0.113.50",  
    "dstuser": "admin"  
  }  
}
```

C.3 Operational Validation Commands

```
# Wazuh service status
sudo systemctl status wazuh-manager --no-pager

# Wazuh configuration syntax test
sudo /var/ossec/bin/wazuh-analysisd -t

# Custom integration files
sudo ls -l /var/ossec/integrations/custom-n8n*

# Integration delivery evidence
sudo tail -n 20 /var/ossec/logs/integrations.log

# Network reachability to n8n
curl -I http://192.168.149.1:5678
```